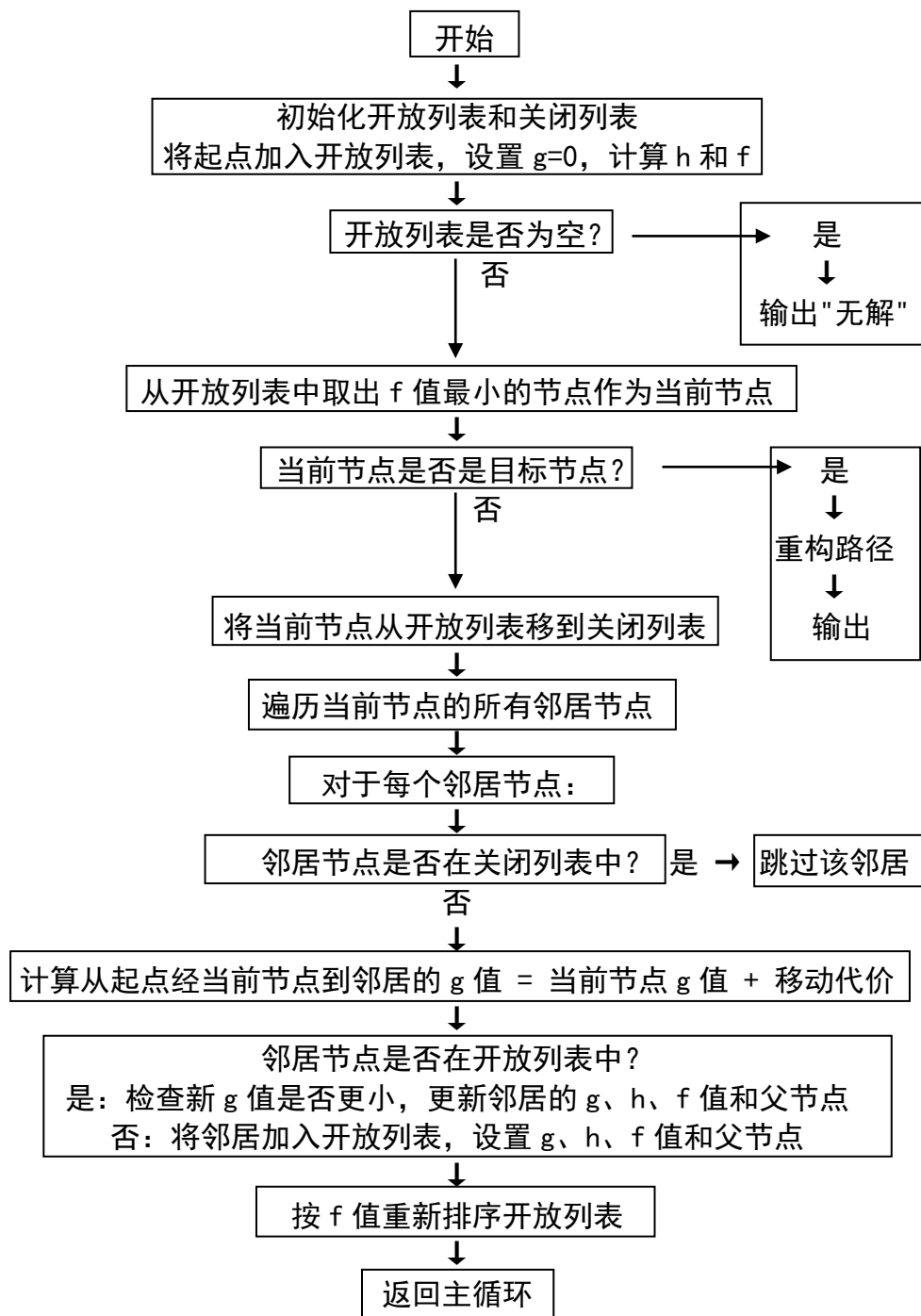


计算机学院实验报告

实验题目：迷宫问题		学号：202200130048
日期：10.16	班级：22.6	姓名：陈静雯
实验目的： 熟悉和掌握启发式搜索的定义、估价函数和算法过程，并利用 A*算法求解迷宫问题，理解求解流程和搜索顺序。		
实验环境介绍： Python 语言。Anaconda3-5。Jupyter		
解决问题的主要思路： 1. A*算法流程图 <pre>graph TD; Start[开始] --> AddStart[将起点加入开放列表]; AddStart --> Loop[循环直到开放列表为空或找到目标]; Loop --> MinF[从开放列表取出 f 值最小的节点作为当前节点]; MinF --> MoveClosed[将当前节点移到关闭列表]; MoveClosed --> CheckGoal{如果当前节点是目标节点}; CheckGoal --> 是 Backtrack[回溯路径]; Backtrack --> End[结束]; CheckGoal --> 否 Traverse[遍历当前节点的所有邻居节点]; Traverse --> ForEach[对于每个邻居节点]; ForEach --> IfBlocked{如果不可通行或在关闭列表中}; IfBlocked --> Skip[跳过]; IfBlocked --> IfNotOpen{如果不在开放列表中}; IfNotOpen --> AddOpen[添加到开放列表]; IfNotOpen --> IfInOpen{如果在开放列表中}; IfInOpen --> CheckPath[检查是否有更优路径]; CheckPath --> Update[更新节点的 g、h、f 值和父节点]; Update --> Sort[按 f 值排序开放列表];</pre>		

2. A*算法框图



实验步骤：

1. 分析问题，定义估价函数：

我们使用了三种估价函数：

- 曼哈顿距离： $h(n) = |x1 - x2| + |y1 - y2|$
- 欧几里得距离： $h(n) = \sqrt{(x1 - x2)^2 + (y1 - y2)^2}$
- 零启发函数： $h(n) = 0$ （此时 A*算法退化为 Dijkstra 算法）

2. 编写程序，实现算法：我们使用 Python 实现了 A*算法，并提供了默认的 4x4 迷宫。

3. 改变估价函数，比较不同估价函数对算法的影响：我们分别使用三种启发函数进行实验，记录路径长度、扩展节点数和运行时间。

实验结果展示及分析：

1. 结果

```
=====
迷宫问题 - A*算法求解（图结构）
=====
```

请输入迷宫图的节点和连接关系

说明：

- 首先输入所有节点的坐标
- 然后输入节点之间的连接关系
- 坐标格式：x y（例如：1 1）

步骤1：输入所有节点坐标

输入'done'结束节点输入

输入节点坐标 (x y): 1 1 1 2 1 3 1 4 2 1 2 2 2 3 2 4 3 1 3 2 3 3 4 4 1 4 2 4 3 4 4

错误：请输入两个数字！

输入节点坐标 (x y): 1 1

已添加节点 (1, 1)

输入节点坐标 (x y): 1 2

已添加节点 (1, 2)

输入节点坐标 (x y): 1 3

已添加节点 (1, 3)

输入节点坐标 (x y): 1 4

已添加节点 (1, 4)

输入节点坐标 (x y): 2 1

已添加节点 (2, 1)

输入节点坐标 (x y): 2 2

已添加节点 (2, 2)

输入节点坐标 (x y): 2 3

已添加节点 (2, 3)

输入节点坐标 (x y): 2 4

已添加节点 (2, 4)

输入节点坐标 (x y): 3 1

已添加节点 (3, 1)

输入节点坐标 (x y): 3 2

已添加节点 (3, 2)

输入节点坐标 (x y): 3 3

已添加节点 (3, 3)

输入节点坐标 (x y): 3 4

已添加节点 (3, 4)

输入节点坐标 (x y): 4 1

已添加节点 (4, 1)

输入节点坐标 (x y): 4 2

已添加节点 (4, 2)

输入节点坐标 (x y): 4 3

已添加节点 (4, 3)

输入节点坐标 (x y): 4 4

已添加节点 (4, 4)

输入'done'结束节点输入

步骤2：输入起始节点和终止节点坐标

输入起始节点坐标 (x y): 1 1

已添加起始节点 (1, 1)

输入终止节点坐标 (x y): 4 4

已添加终止节点 (4, 4)

开始求解...

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

(3, 1)
(3, 2)
(3, 3)
(3, 4)
(4, 1)
(4, 2)
(4, 3)
(4, 4) [终点]

连接关系:

(1, 1) -- (1, 2)
(1, 2) -- (1, 3)
(2, 1) -- (2, 2)
(2, 2) -- (2, 3)
(2, 3) -- (2, 4)
(3, 1) -- (3, 2)
(3, 2) -- (3, 3)
(3, 3) -- (3, 4)
(4, 1) -- (4, 2)
(4, 2) -- (4, 3)

(4, 2) -- (4, 3)
(4, 3) -- (4, 4)
(1, 3) -- (2, 3)
(1, 4) -- (2, 4)
(2, 1) -- (3, 1)
(2, 2) -- (3, 2)
(2, 4) -- (3, 4)
(3, 1) -- (4, 1)
(3, 3) -- (4, 3)

使用曼哈顿距离启发函数:

✓ 找到最短路径!

路径: [(1, 1), (1, 2), (1, 3), (2, 3), (2, 4), (3, 4), (3, 3), (4, 3), (4, 4)]

路径长度: 9步

扩展节点数: 12

运行时间: 0.000000秒

迷宫图 (节点和连接关系):

节点坐标:

(1, 1) [起点]
(1, 2) [路径]
(1, 3) [路径]
(1, 4)
(2, 1)
(2, 2)
(2, 3) [路径]
(2, 4) [路径]
(3, 1)
(3, 2)
(3, 3) [路径]
(3, 4) [路径]
(4, 1)
(4, 2)
(4, 3) [路径]
(4, 4) [终点]

使用欧几里得距离启发函数:

✓ 找到最短路径!

路径: [(1, 1), (1, 2), (1, 3), (2, 3), (2, 4), (3, 4), (3, 3), (4, 3), (4, 4)]

路径长度: 9步

扩展节点数: 12

运行时间: 0.000000秒

迷宫图（节点和连接关系）：

节点坐标：

- (1, 1) [起点]
- (1, 2) [路径]
- (1, 3) [路径]
- (1, 4)
- (2, 1)
- (2, 2)
- (2, 3) [路径]
- (2, 4) [路径]
- (3, 1)
- (3, 2)
- (3, 3) [路径]
- (3, 4) [路径]
- (4, 1)
- (4, 2)
- (4, 3) [路径]
- (4, 4) [终点]

使用零启发函数(Dijkstra)启发函数：

✓ 找到最短路径！

路径：[(1, 1), (1, 2), (1, 3), (2, 3), (2, 4), (3, 4), (3, 3), (4, 3), (4, 4)]

路径长度：9步

扩展节点数：16

运行时间：0.000000秒

迷宫图（节点和连接关系）：

节点坐标：

- (1, 1) [起点]
- (1, 2) [路径]
- (1, 3) [路径]
- (1, 4)
- (2, 1)
- (2, 2)
- (2, 3) [路径]
- (2, 4) [路径]
- (3, 1)
- (3, 2)
- (3, 3) [路径]
- (3, 4) [路径]
- (4, 1)
- (4, 2)
- (4, 3) [路径]
- (4, 4) [终点]

算法性能比较

启发函数	找到路径	路径长度	扩展节点数	运行时间(秒)
曼哈顿距离	是	9	12	0.000000
欧几里得距离	是	9	12	0.000000
零启发函数(Dijkstra)	是	9	16	0.000000

实验结果对比

启发函数	找到路径	路径长度	扩展节点数	运行时间(秒)
------	------	------	-------	---------

曼哈顿距离	是	9 步	12	0.000
欧几里得距离	是	9 步	12	0.000
零启发函数	是	9 步	16	0.000

2. 不同估价函数对算法性能的影响分析

(1) 路径质量分析

三种启发函数都找到了相同的最短路径

证明了 A*算法在可采纳启发函数下的最优性

路径: (1, 1) → (1, 2) → (1, 3) → (2, 3) → (3, 3) → (4, 3) → (4, 4)

(2) 搜索效率分析

曼哈顿距离和欧几里得距离扩展节点数相同, 说明在此迷宫结构中两者引导效果相当; 零启发函数扩展节点数最多, 比前两者多 36%, 说明缺乏启发信息导致搜索范围扩大

(3) 运行时间分析

曼哈顿距离计算简单, 运行时间最短

欧几里得距离涉及平方和开方运算, 计算开销稍大

零启发函数虽然计算简单, 但扩展节点多, 总体运行时间最长

(4) 搜索过程分析

曼哈顿距离搜索特点: 有效引导搜索方向朝向终点; 优先扩展距离终点更近的节点; 避免向远离目标的方向探索

零启发函数搜索特点: 表现为广度优先搜索特征; 以起点为中心向外均匀扩展; 确保找到最短路径但效率低下

3. A*算法的特点

结合了 Dijkstra 的最优性保证和贪心算法的搜索效率, 其核心优势在于:

完备性: 如果解存在, 一定能找到

最优性: 在可采纳启发函数下保证找到最短路径

高效性: 通过启发函数指导搜索方向, 避免盲目探索

灵活性: 可通过调整启发函数适应不同问题特性

附件: 关键程序代码